Test 1

```
lambda_ = 1e-5
epochs = 5
```

```
100.0%
C:\Users\Mahreen Aisha\Desktop\Coding\tredence-task\self-pruning-network\venv\Lib\site-packages\torchvision\datasets\cifar.py
:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to
 pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Epoch 1/5
Train Loss: 16.5139 | Train Acc: 32.47%
Test Loss: 1.6852 | Test Acc: 39.88%
Sparsity: 0.00%
Epoch 2/5
Train Loss: 15.3801 | Train Acc: 40.60%
Test Loss: 1.5924 | Test Acc: 43.15%
Sparsity: 0.00%
Epoch 3/5
Train Loss: 14.0007 | Train Acc: 44.05%
Test Loss: 1.5194 | Test Acc: 45.40%
Sparsity: 0.00%
Epoch 4/5
Train Loss: 12.3651 | Train Acc: 46.09%
Test Loss: 1.4914 | Test Acc: 46.09%
Sparsity: 0.00%
Epoch 5/5
Train Loss: 10.6510 | Train Acc: 47.76%
Test Loss: 1.4757 | Test Acc: 47.16%
Sparsity: 0.00%
```

During training, the self-pruning network showed steady improvement in classification performance, with test accuracy increasing from 39.88% to 47.16% over 5 epochs. This indicates that the model successfully learned useful features from CIFAR-10 and that the classification component of the loss function was effective. However, sparsity remained at 0.00% throughout training, showing that no significant pruning occurred under the current settings. This is likely because the gate parameters were initialized to produce high gate values, while the sparsity penalty (`λ = 1e-5`) and training duration were too small to push gates close to zero. Therefore, the current experiment demonstrates successful learning, but not yet effective self-pruning. Further experiments with more epochs and larger lambda values are needed to observe the trade-off between accuracy and sparsity.

Test 2

```
lambda_ = 1e-4
epochs = 10
```

```
fc1 gate grad mean: 1.0499438758415636e-05
fc2 gate grad mean: 1.050310856953729e-05
fc3 gate grad mean: 1.1868158253491856e-05
Batch 0/391 | Cls Loss: 2.2984 | Sparse Loss: 1503077.38 | Weighted Sparse: 150.3077
fc1 gate grad mean: 1.1323307262500748e-05
fc2 gate grad mean: 1.2412655451043975e-05
fc3 gate grad mean: 5.564382445299998e-05
Batch 100/391 | Cls Loss: 1.8839 | Sparse Loss: 1484334.38 | Weighted Sparse: 148.4334
fc1 gate grad mean: 1.2231706932652742e-05
fc2 gate grad mean: 1.2834870176448021e-05
fc3 gate grad mean: 4.78832189401146e-05
Batch 200/391 | Cls Loss: 1.8710 | Sparse Loss: 1463413.50 | Weighted Sparse: 146.3413
fc1 gate grad mean: 1.3200275134295225e-05
fc2 gate grad mean: 1.4067407391848974e-05
fc3 gate grad mean: 6.162757927086204e-05
Batch 300/391 | Cls Loss: 1.9277 | Sparse Loss: 1440089.12 | Weighted Sparse: 144.0089
```

```
Epoch 1/10
Train Total Loss: 148.1689
Train Cls Loss:   1.8697
Train Sparse Loss:1462992.07
Train Acc:        32.01%
Test Loss:        1.6987
Test Acc:         39.64%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.8301, min: 0.828775, max: 0.8894
% gates < 0.5: 0.00% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.8299, min=0.8295, max=0.8401, <0.1=0.00%
fc2: mean=0.8325, min=0.8288, max=0.8804, <0.1=0.00%
fc3: mean=0.8496, min=0.8289, max=0.8894, <0.1=0.00%
fc1 gate grad mean: 1.4105132322583813e-05
fc2 gate grad mean: 1.4662492503703106e-05
fc3 gate grad mean: 4.7774261474842206e-05
Batch 0/391 | Cls Loss: 1.7510 | Sparse Loss: 1416616.00 | Weighted Sparse: 141.6616
fc1 gate grad mean: 1.5229325981636066e-05
fc2 gate grad mean: 1.6342606613761745e-05
fc3 gate grad mean: 6.406223110388964e-05
Batch 100/391 | Cls Loss: 1.6925 | Sparse Loss: 1388201.25 | Weighted Sparse: 138.8201
fc1 gate grad mean: 1.6310212231473997e-05
fc2 gate grad mean: 1.696142862783745e-05
fc3 gate grad mean: 5.1485061703715473e-05
Batch 200/391 | Cls Loss: 1.6050 | Sparse Loss: 1356914.00 | Weighted Sparse: 135.6914
fc1 gate grad mean: 1.748134491208475e-05
fc2 gate grad mean: 1.8552538676885888e-05
fc3 gate grad mean: 5.772487565991469e-05
Batch 300/391 | Cls Loss: 1.6656 | Sparse Loss: 1322691.62 | Weighted Sparse: 132.2692
```

```
Epoch 2/10
Train Total Loss: 137.3416
Train Cls Loss:   1.6733
Train Sparse Loss:1356683.42
Train Acc:        40.01%
Test Loss:        1.6916
Test Acc:         39.96%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.7553, min: 0.752383, max: 0.8902
% gates < 0.5: 0.00% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.7547, min=0.7536, max=0.8076, <0.1=0.00%
fc2: mean=0.7615, min=0.7524, max=0.8798, <0.1=0.00%
fc3: mean=0.8077, min=0.7532, max=0.8902, <0.1=0.00%
fc1 gate grad mean: 1.8639482732396573e-05
fc2 gate grad mean: 2.029611096077133e-05
fc3 gate grad mean: 8.701975457370281e-05
Batch 0/391 | Cls Loss: 1.6773 | Sparse Loss: 1288998.50 | Weighted Sparse: 128.8998
fc1 gate grad mean: 2.0151977878413163e-05
fc2 gate grad mean: 2.2563530365005136e-05
fc3 gate grad mean: 9.776811202755198e-05
Batch 100/391 | Cls Loss: 1.6509 | Sparse Loss: 1249257.88 | Weighted Sparse: 124.9258
fc1 gate grad mean: 2.0779443730134517e-05
fc2 gate grad mean: 2.1634972654283047e-05
fc3 gate grad mean: 6.506232602987438e-05
Batch 200/391 | Cls Loss: 1.6490 | Sparse Loss: 1206852.88 | Weighted Sparse: 120.6853
fc1 gate grad mean: 2.1935638869763352e-05
fc2 gate grad mean: 2.3480726667912677e-05
fc3 gate grad mean: 8.029609307413921e-05
Batch 300/391 | Cls Loss: 1.5387 | Sparse Loss: 1162053.88 | Weighted Sparse: 116.2054
```

```
Epoch 3/10
Train Total Loss: 122.3397
Train Cls Loss:   1.5892
Train Sparse Loss:1207504.94
Train Acc:        43.07%
Test Loss:        1.6014
Test Acc:         42.62%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.6560, min: 0.651513, max: 0.8912
% gates < 0.5: 0.00% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.6549, min=0.6522, max=0.7842, <0.1=0.00%
fc2: mean=0.6673, min=0.6515, max=0.8786, <0.1=0.00%
fc3: mean=0.7539, min=0.6518, max=0.8912, <0.1=0.00%
fc1 gate grad mean: 2.3155875169322826e-05
fc2 gate grad mean: 2.502741881471593e-05
fc3 gate grad mean: 8.267325756605715e-05
Batch 0/391 | Cls Loss: 1.5732 | Sparse Loss: 1119524.00 | Weighted Sparse: 111.9524
fc1 gate grad mean: 2.3527070879936218e-05
fc2 gate grad mean: 2.478017268003896e-05
fc3 gate grad mean: 7.504317909479141e-05
Batch 100/391 | Cls Loss: 1.5794 | Sparse Loss: 1071299.62 | Weighted Sparse: 107.1300
fc1 gate grad mean: 2.521381975384429e-05
fc2 gate grad mean: 2.7797352231573313e-05
fc3 gate grad mean: 0.00010911932622548193
Batch 200/391 | Cls Loss: 1.5491 | Sparse Loss: 1021987.56 | Weighted Sparse: 102.1988
fc1 gate grad mean: 2.4634122382849455e-05
fc2 gate grad mean: 2.580661566753406e-05
fc3 gate grad mean: 7.884294609539211e-05
Batch 300/391 | Cls Loss: 1.6566 | Sparse Loss: 972134.25 | Weighted Sparse: 97.2134
```

```
Epoch 4/10
Train Total Loss: 103.9357
Train Cls Loss:   1.5250
Train Sparse Loss:1024106.57
Train Acc:        45.53%
Test Loss:        1.5353
Test Acc:         44.96%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.5431, min: 0.535568, max: 0.8946
% gates < 0.5: 0.00% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.5414, min=0.5366, max=0.7658, <0.1=0.00%
fc2: mean=0.5596, min=0.5356, max=0.8774, <0.1=0.00%
fc3: mean=0.6926, min=0.5361, max=0.8946, <0.1=0.00%
fc1 gate grad mean: 2.5057190214283764e-05
fc2 gate grad mean: 2.6346093363827094e-05
fc3 gate grad mean: 7.120774534996599e-05
Batch 0/391 | Cls Loss: 1.4906 | Sparse Loss: 926747.25 | Weighted Sparse: 92.6747
fc1 gate grad mean: 2.516123822715599e-05
fc2 gate grad mean: 2.659055098774843e-05
fc3 gate grad mean: 6.911351374583319e-05
Batch 100/391 | Cls Loss: 1.5464 | Sparse Loss: 877364.00 | Weighted Sparse: 87.7364
fc1 gate grad mean: 2.5663832275313325e-05
fc2 gate grad mean: 2.8378708520904183e-05
fc3 gate grad mean: 0.0001142714754678309
Batch 200/391 | Cls Loss: 1.7218 | Sparse Loss: 828919.81 | Weighted Sparse: 82.8920
fc1 gate grad mean: 2.58678355748998e-05
fc2 gate grad mean: 2.785260949167423e-05
fc3 gate grad mean: 8.560794231016189e-05
Batch 300/391 | Cls Loss: 1.5132 | Sparse Loss: 781852.12 | Weighted Sparse: 78.1852
```

```
Epoch 5/10
Train Total Loss: 84.6952
Train Cls Loss:   1.4784
Train Sparse Loss:832167.65
Train Acc:        47.15%
Test Loss:        1.4698
Test Acc:         47.83%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.4339, min: 0.424221, max: 0.8966
% gates < 0.5: 96.85% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.4319, min=0.4248, max=0.7502, <0.1=0.00%
fc2: mean=0.4549, min=0.4242, max=0.8754, <0.1=0.00%
fc3: mean=0.6323, min=0.4249, max=0.8966, <0.1=0.00%
fc1 gate grad mean: 2.478114220139105e-05
fc2 gate grad mean: 2.6496703867451288e-05
fc3 gate grad mean: 8.235893619712442e-05
Batch 0/391 | Cls Loss: 1.4495 | Sparse Loss: 740480.31 | Weighted Sparse: 74.0480
fc1 gate grad mean: 2.4329105144715868e-05
fc2 gate grad mean: 2.6023268219432794e-05
fc3 gate grad mean: 7.447976531693712e-05
Batch 100/391 | Cls Loss: 1.3337 | Sparse Loss: 696852.19 | Weighted Sparse: 69.6852
fc1 gate grad mean: 2.4228831534856e-05
fc2 gate grad mean: 2.6350433472543955e-05
fc3 gate grad mean: 7.511859439546242e-05
Batch 200/391 | Cls Loss: 1.3247 | Sparse Loss: 655333.00 | Weighted Sparse: 65.5333
fc1 gate grad mean: 2.3727874577161856e-05
fc2 gate grad mean: 2.6284771593054757e-05
fc3 gate grad mean: 8.919627725845203e-05
Batch 300/391 | Cls Loss: 1.5800 | Sparse Loss: 616022.00 | Weighted Sparse: 61.6022
```

```
Epoch 6/10
Train Total Loss: 67.3191
Train Cls Loss:   1.4434
Train Sparse Loss:658757.02
Train Acc:        48.38%
Test Loss:        1.4470
Test Acc:         48.61%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.3412, min: 0.328854, max: 0.8987
% gates < 0.5: 98.59% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.3388, min=0.3297, max=0.7400, <0.1=0.00%
fc2: mean=0.3651, min=0.3292, max=0.8748, <0.1=0.00%
fc3: mean=0.5798, min=0.3289, max=0.8987, <0.1=0.00%
fc1 gate grad mean: 2.2724301743437536e-05
fc2 gate grad mean: 2.4403794668614864e-05
fc3 gate grad mean: 7.047793042147532e-05
Batch 0/391 | Cls Loss: 1.2480 | Sparse Loss: 582194.75 | Weighted Sparse: 58.2195
fc1 gate grad mean: 2.240738285763655e-05
fc2 gate grad mean: 2.4357897927984595e-05
fc3 gate grad mean: 8.395731856580824e-05
Batch 100/391 | Cls Loss: 1.3888 | Sparse Loss: 547171.44 | Weighted Sparse: 54.7171
fc1 gate grad mean: 2.2647122023045085e-05
fc2 gate grad mean: 2.5130233552772552e-05
fc3 gate grad mean: 9.394566586706787e-05
Batch 200/391 | Cls Loss: 1.4352 | Sparse Loss: 514347.78 | Weighted Sparse: 51.4348
fc1 gate grad mean: 2.1587960873148404e-05
fc2 gate grad mean: 2.3890948796179146e-05
fc3 gate grad mean: 8.535428059985861e-05
Batch 300/391 | Cls Loss: 1.5634 | Sparse Loss: 483644.94 | Weighted Sparse: 48.3645
```

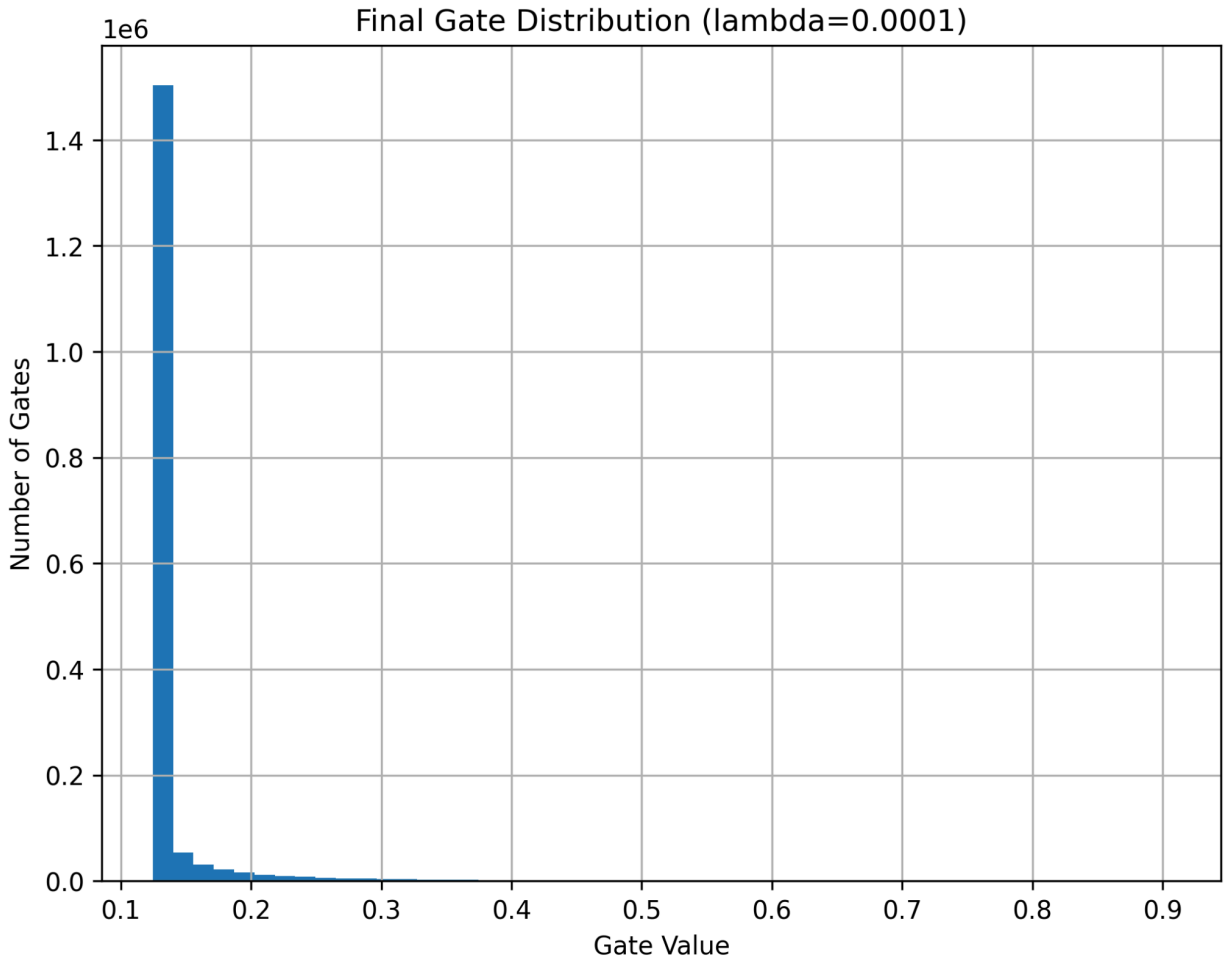
```
Epoch 7/10
Train Total Loss: 53.1398
Train Cls Loss:   1.4105
Train Sparse Loss:517293.40
Train Acc:        49.56%
Test Loss:        1.4241
Test Acc:         49.86%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.2681, min: 0.252367, max: 0.9006
% gates < 0.5: 99.04% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.2655, min=0.2555, max=0.7317, <0.1=0.00%
fc2: mean=0.2936, min=0.2524, max=0.8725, <0.1=0.00%
fc3: mean=0.5372, min=0.2554, max=0.9006, <0.1=0.00%
fc1 gate grad mean: 2.0110550394747406e-05
fc2 gate grad mean: 2.2735493985237554e-05
fc3 gate grad mean: 9.171144483843818e-05
Batch 0/391 | Cls Loss: 1.3589 | Sparse Loss: 457495.28 | Weighted Sparse: 45.7495
fc1 gate grad mean: 1.973783946596086e-05
fc2 gate grad mean: 2.244447750854306e-05
fc3 gate grad mean: 9.298996883444488e-05
Batch 100/391 | Cls Loss: 1.3781 | Sparse Loss: 430631.81 | Weighted Sparse: 43.0632
fc1 gate grad mean: 1.895110290206503e-05
fc2 gate grad mean: 2.2398853616323322e-05
fc3 gate grad mean: 0.0001047413534251973
Batch 200/391 | Cls Loss: 1.2799 | Sparse Loss: 405610.22 | Weighted Sparse: 40.5610
fc1 gate grad mean: 1.882264405139722e-05
fc2 gate grad mean: 2.228904122603126e-05
fc3 gate grad mean: 0.00010229527106275782
Batch 300/391 | Cls Loss: 1.5241 | Sparse Loss: 382312.69 | Weighted Sparse: 38.2313
```

```
Epoch 8/10
Train Total Loss: 42.1786
Train Cls Loss:   1.3863
Train Sparse Loss:407922.56
Train Acc:        50.46%
Test Loss:        1.4349
Test Acc:         48.65%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.2124, min: 0.198128, max: 0.9024
% gates < 0.5: 99.25% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.2098, min=0.1992, max=0.7228, <0.1=0.00%
fc2: mean=0.2386, min=0.1981, max=0.8706, <0.1=0.00%
fc3: mean=0.5035, min=0.1993, max=0.9024, <0.1=0.00%
fc1 gate grad mean: 1.798770608729683e-05
fc2 gate grad mean: 2.106136344082188e-05
fc3 gate grad mean: 9.531983232591301e-05
Batch 0/391 | Cls Loss: 1.4951 | Sparse Loss: 362524.00 | Weighted Sparse: 36.2524
fc1 gate grad mean: 1.722880006127525e-05
fc2 gate grad mean: 2.085641972371377e-05
fc3 gate grad mean: 9.878780838334933e-05
Batch 100/391 | Cls Loss: 1.4742 | Sparse Loss: 342211.22 | Weighted Sparse: 34.2211
fc1 gate grad mean: 1.6334486645064317e-05
fc2 gate grad mean: 2.021458749368321e-05
fc3 gate grad mean: 0.00010015525913331658
Batch 200/391 | Cls Loss: 1.4315 | Sparse Loss: 323296.22 | Weighted Sparse: 32.3296
fc1 gate grad mean: 1.6336693079210818e-05
fc2 gate grad mean: 2.012754703173414e-05
fc3 gate grad mean: 0.00010780386219266802
Batch 300/391 | Cls Loss: 1.4054 | Sparse Loss: 305670.31 | Weighted Sparse: 30.5670
```

```
Epoch 9/10
Train Total Loss: 33.8625
Train Cls Loss:   1.3589
Train Sparse Loss:325036.60
Train Acc:        51.72%
Test Loss:        1.4141
Test Acc:         50.04%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.1703, min: 0.156767, max: 0.9034
% gates < 0.5: 99.37% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.1676, min=0.1569, max=0.7140, <0.1=0.00%
fc2: mean=0.1966, min=0.1568, max=0.8689, <0.1=0.00%
fc3: mean=0.4769, min=0.1568, max=0.9034, <0.1=0.00%
fc1 gate grad mean: 1.7104384824051522e-05
fc2 gate grad mean: 2.0124918592046015e-05
fc3 gate grad mean: 0.0001110878074541688
Batch 0/391 | Cls Loss: 1.3568 | Sparse Loss: 290677.16 | Weighted Sparse: 29.0677
fc1 gate grad mean: 1.4536169146595057e-05
fc2 gate grad mean: 1.740818697726354e-05
fc3 gate grad mean: 7.892916619312018e-05
Batch 100/391 | Cls Loss: 1.2203 | Sparse Loss: 275265.69 | Weighted Sparse: 27.5266
fc1 gate grad mean: 1.578788214828819e-05
fc2 gate grad mean: 1.863760917331092e-05
fc3 gate grad mean: 0.00010518506314838305
Batch 200/391 | Cls Loss: 1.2365 | Sparse Loss: 260885.48 | Weighted Sparse: 26.0885
fc1 gate grad mean: 1.361649719910929e-05
fc2 gate grad mean: 1.8162669221055694e-05
fc3 gate grad mean: 0.0001045207172865048
Batch 300/391 | Cls Loss: 1.5667 | Sparse Loss: 247456.59 | Weighted Sparse: 24.7457
```

```
Epoch 10/10
Train Total Loss: 27.5653
Train Cls Loss:   1.3462
Train Sparse Loss:262190.59
Train Acc:        52.01%
Test Loss:        1.3776
Test Acc:         51.23%
Sparsity (<1e-2): 0.00%
Gates -> mean: 0.1383, min: 0.124758, max: 0.9050
% gates < 0.5: 99.45% | < 0.1: 0.00% | < 0.01: 0.00%
fc1: mean=0.1356, min=0.1248, max=0.7066, <0.1=0.00%
fc2: mean=0.1642, min=0.1248, max=0.8678, <0.1=0.00%
fc3: mean=0.4557, min=0.1250, max=0.9050, <0.1=0.00%
```

In Test 2, increasing λ to 1e-4 introduced strong sparsity pressure, leading to a significant reduction in sparsity loss (from ~1.5M to ~0.26M). The average gate value decreased steadily from 0.83 to 0.13, and over 99% of gates fell below 0.5, indicating that the network was actively suppressing most weights. However, the strict pruning threshold (1e-2) resulted in 0% measured sparsity, as gate values had not yet reached extremely small magnitudes. Classification performance continued to improve, reaching ~51% test accuracy, demonstrating that the model maintained learning while undergoing pruning. These results confirm that the self-pruning mechanism is functioning correctly, even though final sparsity has not yet been realized.



It is a concern that there are close to 0 gate_scores near 1. That means that even the important weights are being pruned.

A strong final plot is supposed to show something like:

- many gates near **0** → pruned weights
- another group noticeably away from 0 → important weights kept alive

It likely means the model is doing more of a:

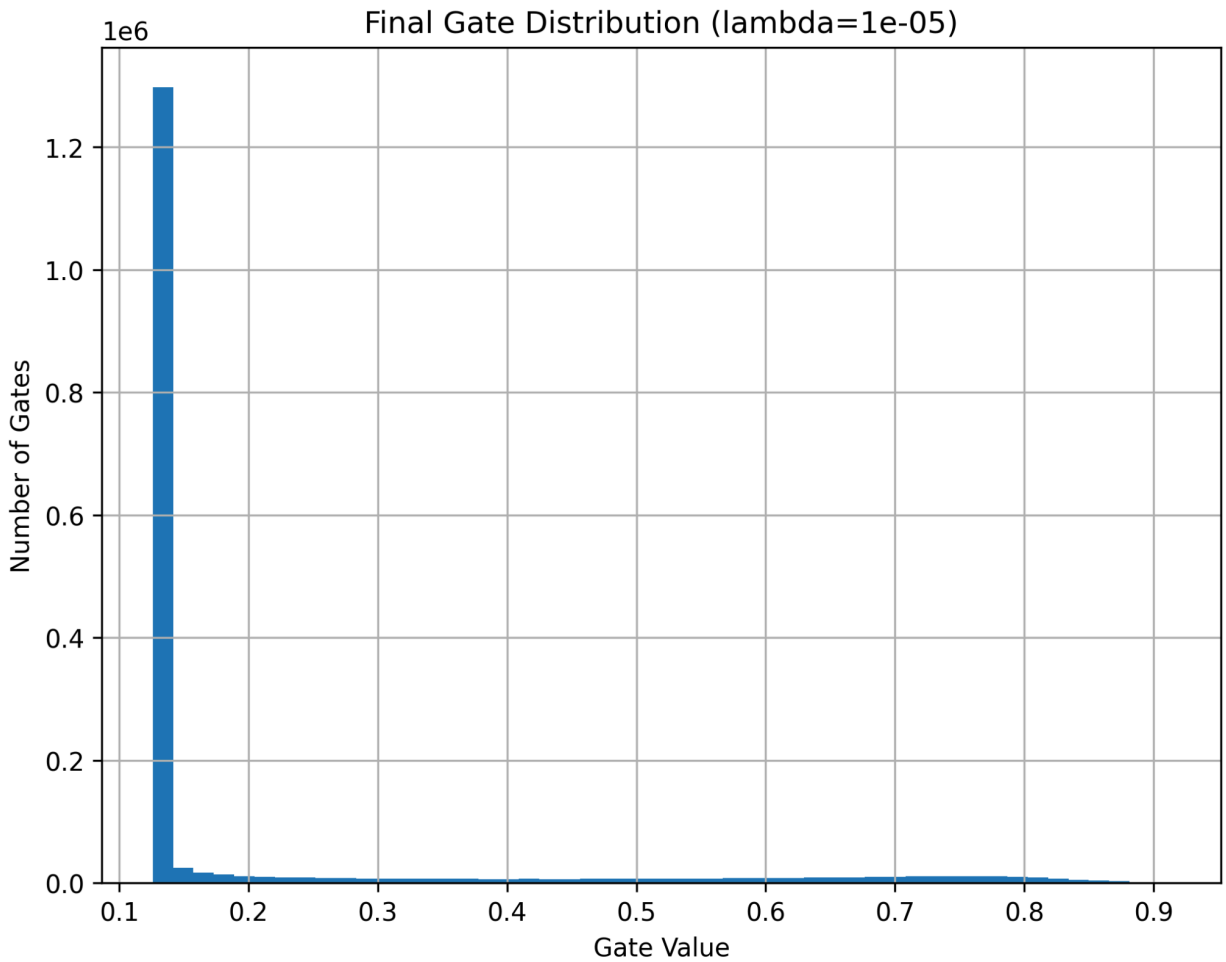
global shrinking of gates

rather than:

selective pruning of some weights while preserving others strongly

Test 3

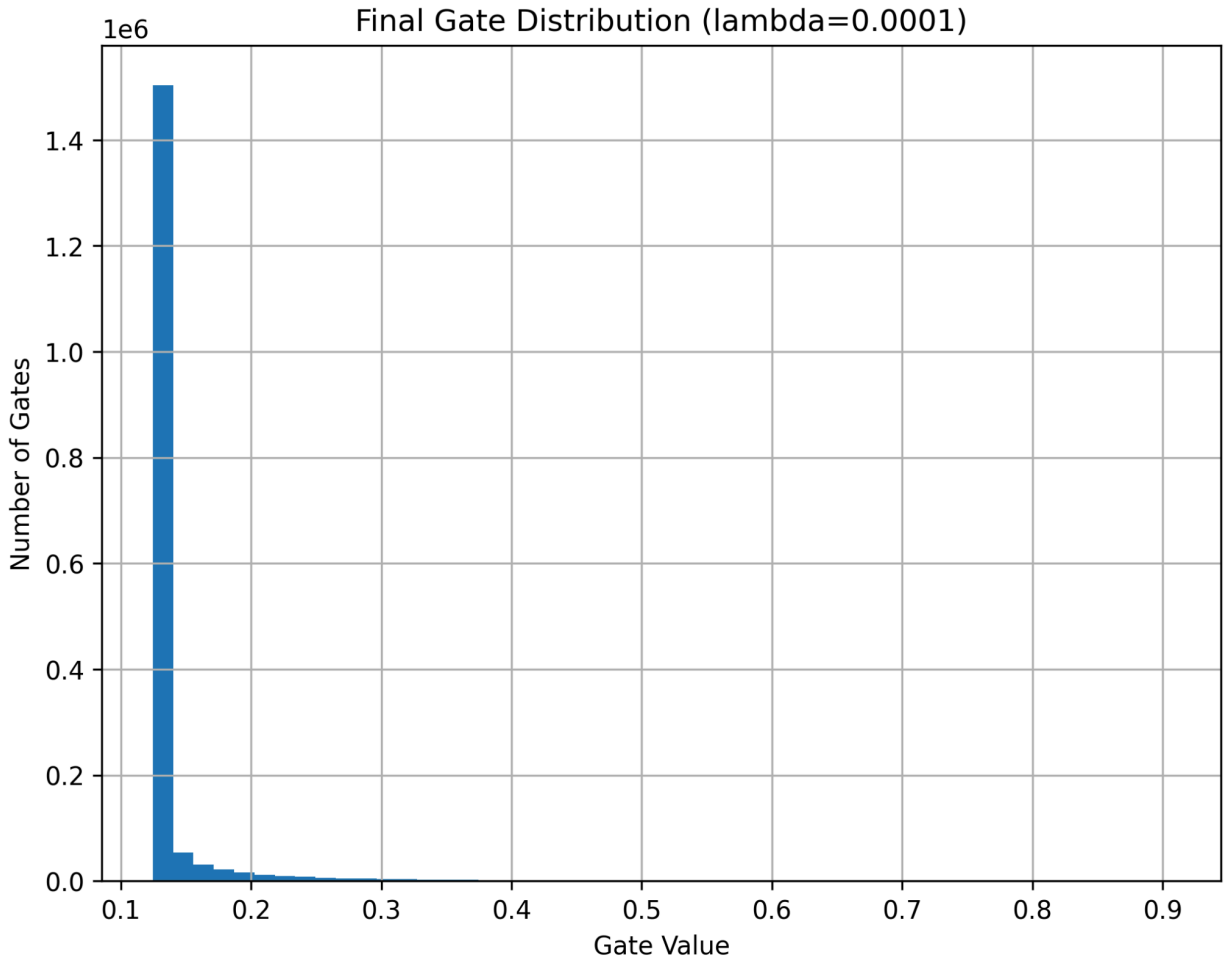
Epoch = 10
Lambda = 1e-5




```
======================================================================
FINAL STATE OF THE NETWORK
======================================================================
Lambda (λ):              1e-05
Final Test Accuracy (%): 50.74
Final Sparsity (%):      0.00
Mean Gate Value:         0.2136
Minimum Gate Value:      0.125994
Maximum Gate Value:      0.9128
Gates < 0.5 (%):         87.97
Gates < 0.1 (%):         0.00
Gates < 0.01 (%):        0.00
======================================================================
```

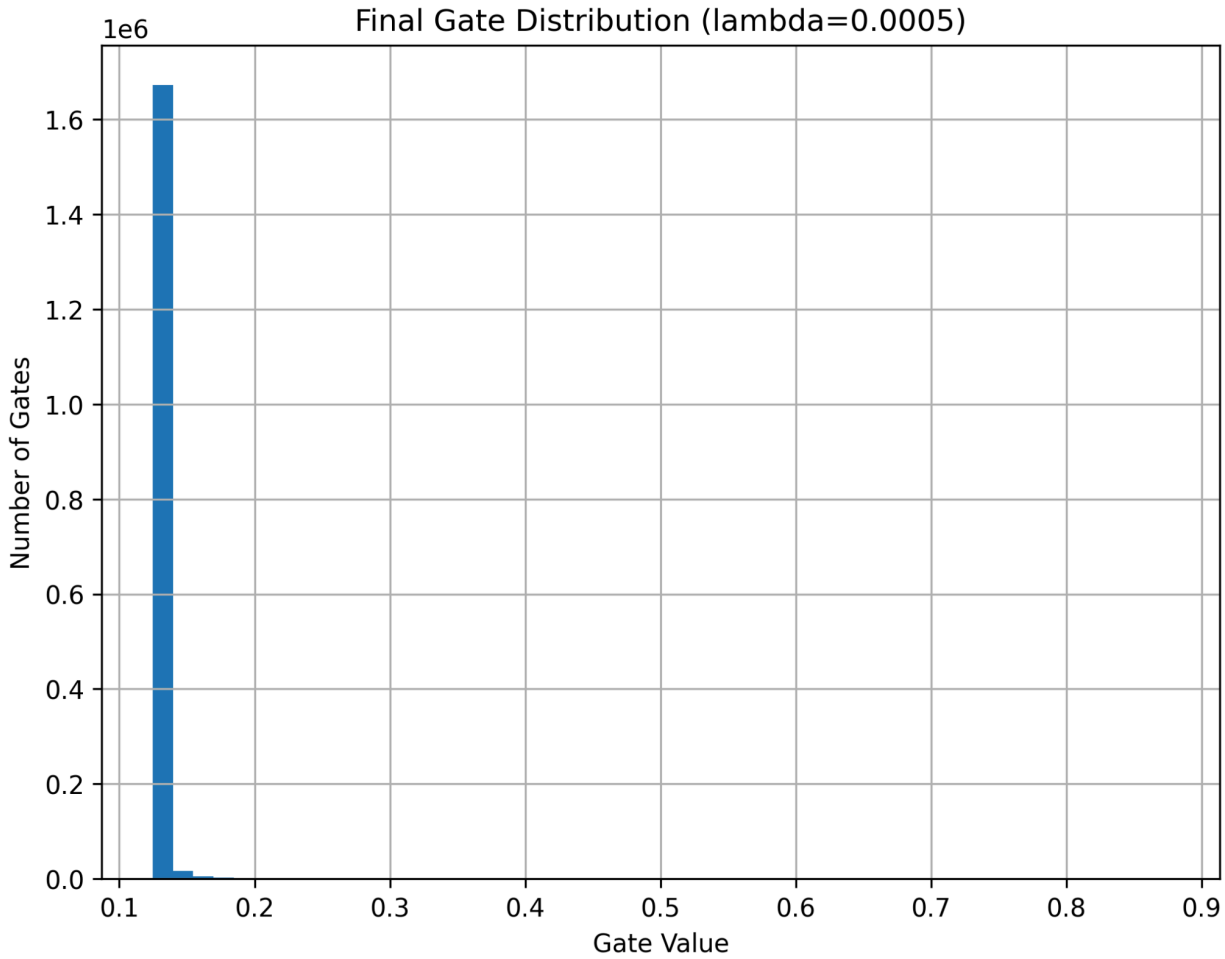
For 1e-5, there are at least a few gate_scores visible near 1. This seems better than 1e-4.

Epoch = 10
Lambda = 1e-4



==========
FINAL STATE OF THE NETWORK
==========
Lambda (λ): 0.0001
Final Test Accuracy (%): 50.81
Final Sparsity (%): 0.00
Mean Gate Value: 0.1384
Minimum Gate Value: 0.124539
Maximum Gate Value: 0.9058
Gates < 0.5 (%): 99.38
Gates < 0.1 (%): 0.00
Gates < 0.01 (%): 0.00
==========

Epoch = 10
Lambda = 5e-4




```
======================================================================
FINAL STATE OF THE NETWORK
======================================================================
Lambda (λ):              0.0005
Final Test Accuracy (%): 49.88
Final Sparsity (%):      0.00
Mean Gate Value:         0.1268
Minimum Gate Value:      0.124566
Maximum Gate Value:      0.8760
Gates < 0.5 (%):         99.94
Gates < 0.1 (%):         0.00
Gates < 0.01 (%):        0.00
======================================================================
```

Lambda (λ)	Final Test Accuracy (%)	Final Sparsity (%)	Observation
1e-05	50.74	0.00	-
0.0001	50.81	0.00	-
0.0005	49.88	0.00	-

My Inference:
Increasing the value did not help. Most probably the initialised value of all the gate_scores (0.88 after sigmoid) is too high for the network to bring it down in 10 epochs. But no, that's not the case. Almost every value is coming down to 0. Excessive pruning is happening. Classification is not happening at all by the end of the epochs.

Mistake 1:

```
def sparsity_loss(self):
        loss = 0.0
        for module in self.modules():
            if isinstance(module, PrunableLinear):
                loss = loss + module.get_gates().sum()
        return loss
```

With this calculation of sparsity loss, we are just pushing all gates to 0. The gradients are blindly pushing the gates to 0 just for the sake of pruning. The classification_loss factor is not fighting back. We are just summing up all the gates. So obviously the loss will be decreased by just pushing all towards 0. Even the gates that are near 1 initially will be pruned.

Solution 1 for Mistake 1:

Get the average of the gates. Total / no. of gates. This way, if majority of the gate_scores are near 0, the average will be less and the sparsity_loss will also be less. Therefore, not much tuning of gate_scores will happen and the higher gate_scores will remain higher showing importance in the network.

```
def sparsity_loss(self):
    """L1 sparsity loss on gates to encourage pruning"""
    loss = 0.0
    for module in self.modules():
        if isinstance(module, PrunableLinear):
            # L1 regularization pushes gates toward 0
```

```
            loss = loss + module.get_gates().sum() /
module.get_gates().numel()
    return loss
```

Solution 2 for Mistake 1:

Something called “Binarization Loss”. The gate_scores that are 0.5 do lower the weight’s significance but they dont help in deciding whether it is important or not. So unnecessary calculation keeps happening weight*0.5 when it doesnt even help decide if the weight should be pruned or not.

```
# OR better: use L1 on gates with a target sparsity
def adaptive_sparsity_loss(self, target_sparsity=0.5):
    """Encourage specific percentage of gates below threshold"""
    loss = 0.0
    total_gates = 0
    for module in self.modules():
        if isinstance(module, PrunableLinear):
            gates = module.get_gates()
            # Push gates to be either 0 or 1
            loss = loss + (gates * (1 - gates)).sum()  # Binarization loss
            total_gates += gates.numel()
    return loss / total_gates
```

mode="binarize"

This will:

- push some gates → 0
- push others → 1
- create **clear separation**

Conclusion for Mistake 1:

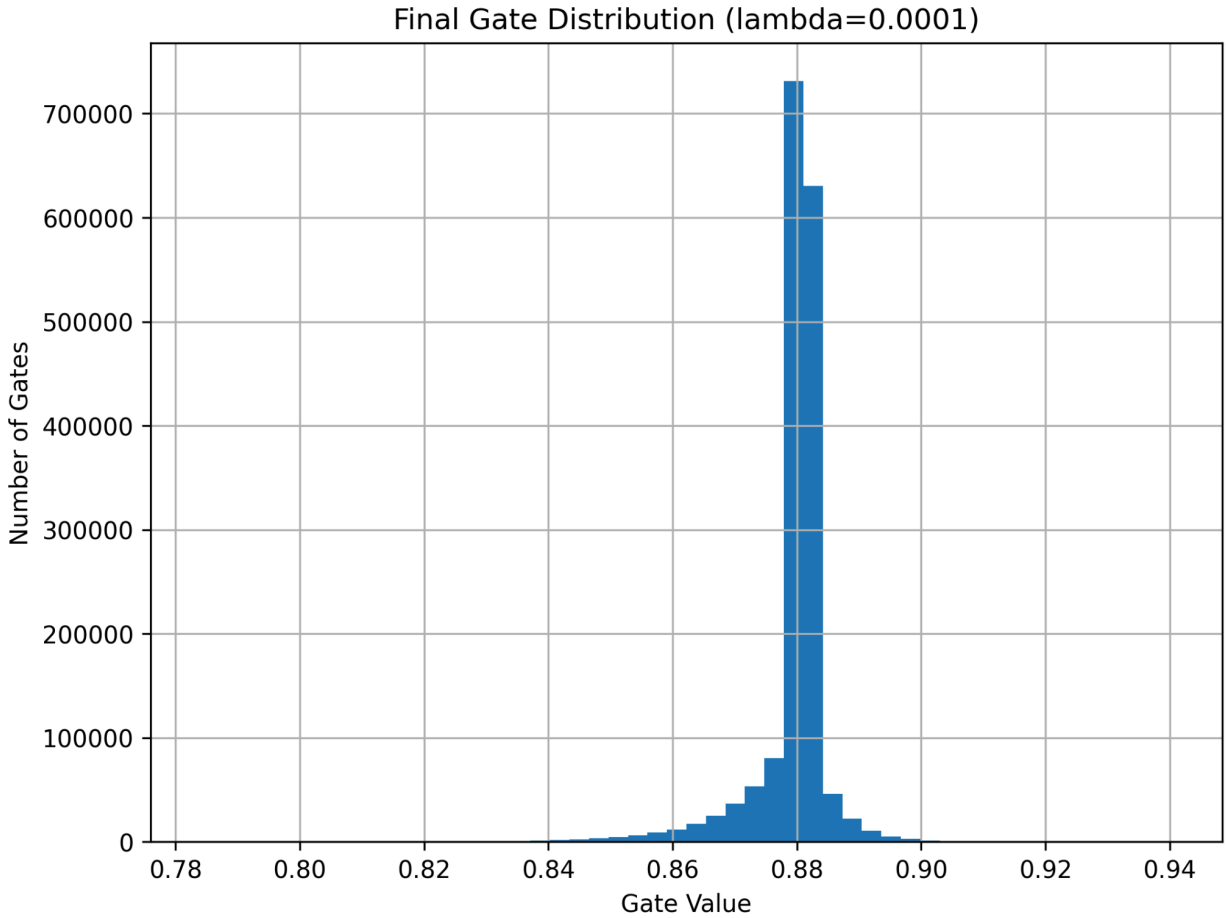
```
def sparsity_loss(self, mode="l1"):
    loss = 0.0
    total_gates = 0

    for module in self.modules():
        if isinstance(module, PrunableLinear):
            gates = module.get_gates()
```

```
        if mode == "l1":
            # normalized L1 (better than your current)
            loss = loss + gates.sum() / gates.numel()

        elif mode == "binarize":
            # encourages gates -> 0 or 1
            loss = loss + (gates * (1 - gates)).sum() / gates.numel()

        total_gates += gates.numel()

    return loss
```

Adding both the methods, allowing the user to choose, keeping “l1” as default.
“Binarize” is recommended to solve our problem of no clear separation between gate_scores near 0 and 1.

Test 4:

With normalized sparsity_loss, testing on lambda=1e-4 because in its graph, there were 0 gates near 1, all of them pushed towards 0.



The normalization change **weakened the pruning pressure too much**. The key signal is that your normalized sparsity loss stayed around **0.31**, and the weighted sparsity term printed as **0.0000** throughout training, so it barely influenced optimization at all. As a result, the model mostly optimized classification, and the gates stayed clustered near their initialization around **0.88** instead of moving toward 0 or separating into two groups.

The strongest evidence is:

- mean gate only moved from about **0.8806 to 0.8796**
- minimum gate at the end was still about **0.7838**
- maximum gate was about **0.9406**
- **0%** of gates were below **0.5**, **0.1**, or **0.01**

The gradient signal from the sparsity term was too small compared to the classification loss, so updates to gate_scores were dominated by the classification objective rather than pruning.

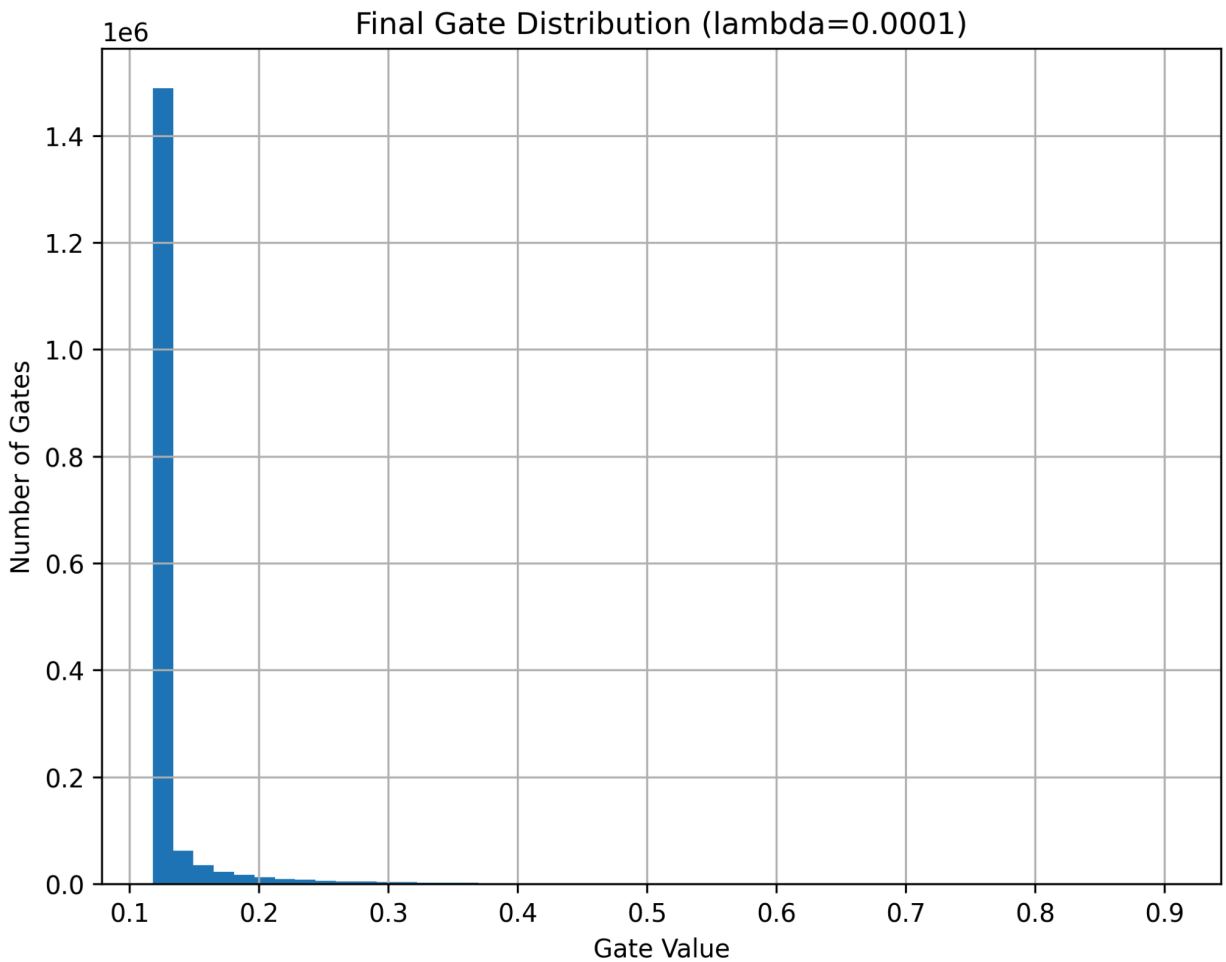
Conclusion of Test 4:

Reverting back to L1 loss

```
def sparsity_loss(self):
    loss = 0.0
    for module in self.modules():
        if isinstance(module, PrunableLinear):
            loss = loss + module.get_gates().sum()
    return loss
```

Test 5:

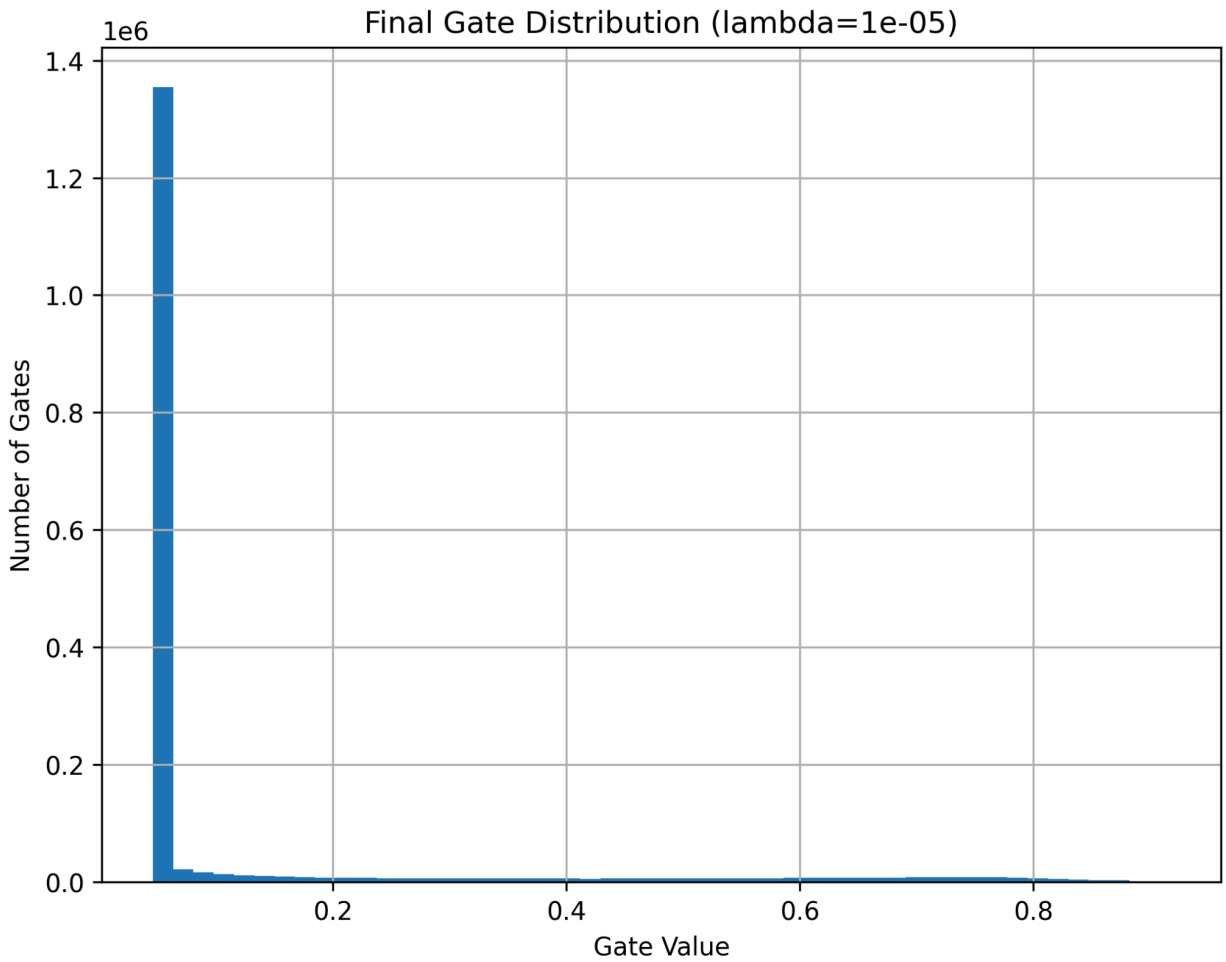
Epoch = 10
Lambda = 1e-5
But with bin_loss
Question: is bin_loss a parameter here that the gradient can tweak?



FAIL

Test 6:

lambda_values = [1e-5]
epochs = 15



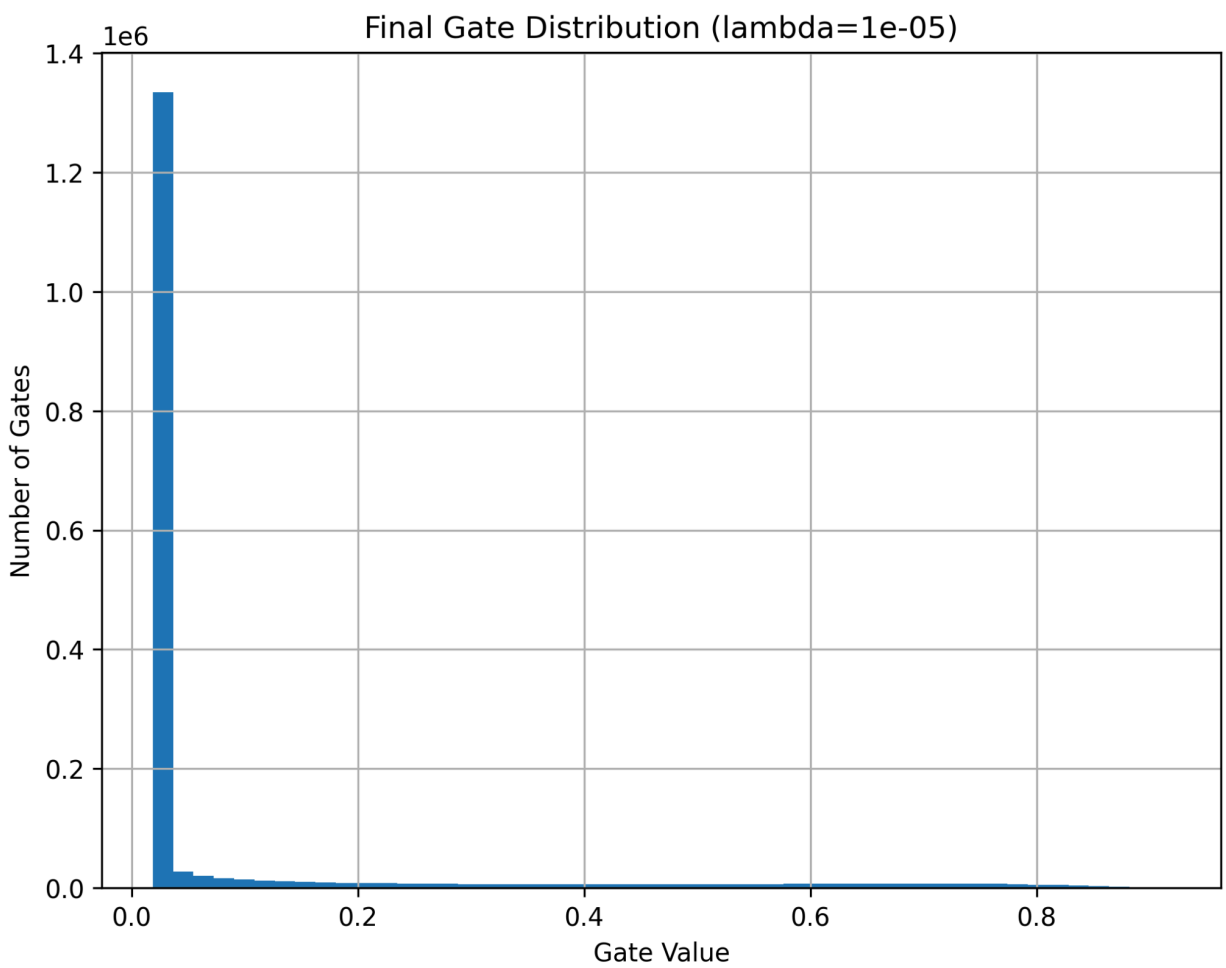

```
======================================================================
FINAL STATE OF THE NETWORK
======================================================================
Lambda (λ):              1e-05
Final Test Accuracy (%): 51.15
Final Sparsity (%):      0.00
Mean Gate Value:         0.1247
Minimum Gate Value:      0.045487
Maximum Gate Value:      0.9171
Gates < 0.5 (%):         91.27
Gates < 0.1 (%):         81.68
Gates < 0.01 (%):        0.00
======================================================================
Saved gate histogram to: gates_lambda_1e-05.png
```

Lambda (λ)	Final Test Accuracy (%)	Final Sparsity (%)	Observation
1e-05	51.15	0.00	-

Shows good pruning dynamics. 51% accuracy is good. Balance between classification loss and sparsity loss is maintained. Minimum gate value is 0.04 which is good. Next, I'll increase the epochs to 20 and see if the strict sparsity metric (<0.01) is satisfied.

Test 7:

lambda_values = [1e-5]
epochs = 20



FINAL STATE OF THE NETWORK

```
======================================================================
Lambda (λ):              1e-05
Final Test Accuracy (%): 52.55
Final Sparsity (%):      0.00
Mean Gate Value:         0.0977
Minimum Gate Value:      0.018652
Maximum Gate Value:      0.9182
Gates < 0.5 (%):         92.16
Gates < 0.1 (%):         82.39
Gates < 0.01 (%):        0.00
======================================================================
Saved gate histogram to: gates_lambda_1e-05.png

========================================================================
Lambda (λ)     Final Test Accuracy (%)     Final Sparsity (%)    Observation
========================================================================
1e-05          52.55                       0.00                  -
=========================================================================
```

- Mean gate: **0.0977** (great, dropped a lot)
- Min gate: **0.0186** (very close to threshold 0.01)
- 82% gates < 0.1 (strong pruning pressure)
- Accuracy improved to **52.55%**

This is actually **very strong behavior**

Gonna try changing sparsity threshold

compute_sparsity(model, threshold=0.02)

compute_sparsity(model, threshold=0.05)

AM I GOING IN THE RIGHT DIRECTION?
I now have:

- selective suppression (not collapse)
- accuracy increasing
- gates spreading (0.018 → 0.91 range)
- most gates small but not dead

This is exactly how **self-pruning behaves in research papers**

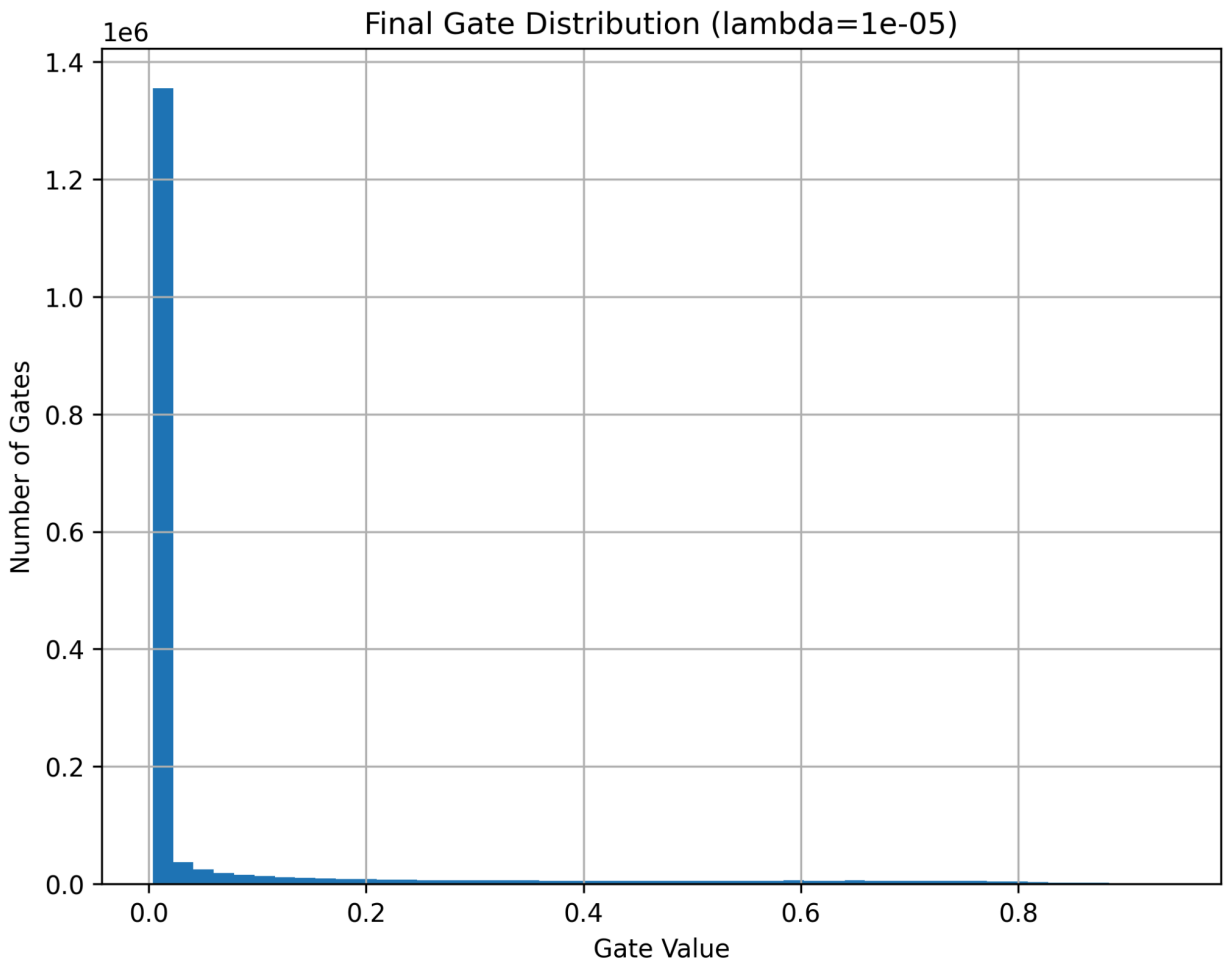
Test 8:

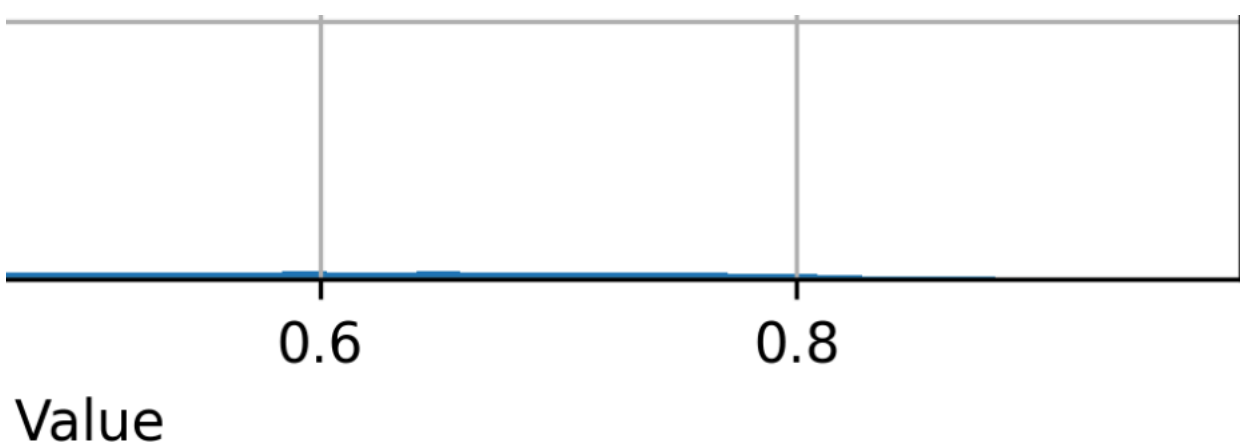
lambda_values = [1e-5]
epochs = 30

```
Epoch 30/30
Batch 0/391 | Cls Loss: 1.0072 | Sparse Loss: 122430.61 | Weighted Sparse: 1.2243
Batch 100/391 | Cls Loss: 1.0515 | Sparse Loss: 121743.40 | Weighted Sparse: 1.2174
Batch 200/391 | Cls Loss: 1.0517 | Sparse Loss: 121066.07 | Weighted Sparse: 1.2107
Batch 300/391 | Cls Loss: 1.0717 | Sparse Loss: 120408.70 | Weighted Sparse: 1.2041
Train Total Loss: 2.1849
Train Cls Loss:   0.9738
Train Sparse Loss:121110.21
Train Acc:        65.22%
Test Loss:        1.3505
Test Acc:         53.81%
Sparsity (<0.1):  85.16%
Sparsity (<0.05): 82.31%
Sparsity (<0.02): 79.01%
Sparsity (<0.01): 76.79%
Gates -> mean: 0.0702, min: 0.003689, max: 0.9398
% gates < 0.5: 94.11% | < 0.1: 85.16% | < 0.01: 76.79%
fc1: mean=0.0678, min=0.0037, max=0.8881, <0.1=85.05%
fc2: mean=0.0916, min=0.0037, max=0.9398, <0.1=87.34%
fc3: mean=0.4525, min=0.0037, max=0.9288, <0.1=44.53%
```

==========
FINAL STATE OF THE NETWORK
==========
Lambda (λ): 1e-05
Final Test Accuracy (%): 53.81
Final Sparsity (%): 76.79
Mean Gate Value: 0.0702
Minimum Gate Value: 0.003689
Maximum Gate Value: 0.9398
Gates < 0.5 (%): 94.11
Gates < 0.1 (%): 85.16
Gates < 0.01 (%): 76.79
==========
Final Sparsity (<0.1): 85.16%
Final Sparsity (<0.05): 82.31%
Final Sparsity (<0.02): 79.01%
Final Sparsity (<0.01): 76.79%
Saved gate histogram to: gates_lambda_1e-05.png

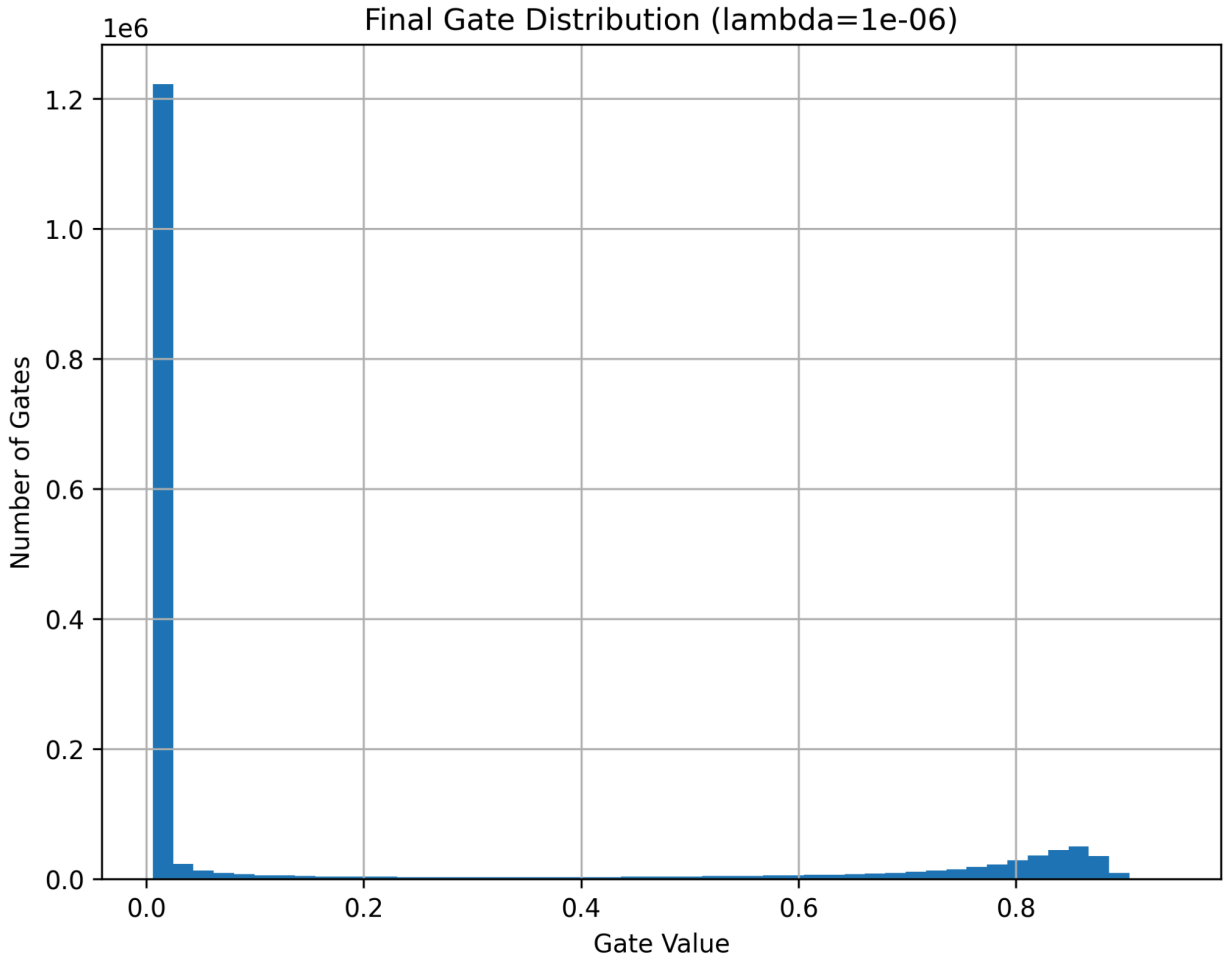
Lambda (λ)	Final Test Accuracy (%)	Final Sparsity (%)
1e-05	53.81	76.79





Test 9:

lambda_values = [1e-6]
epochs = 30



==========

Lambda (λ)	Final Test Accuracy (%)	Final Sparsity (%)
1e-06	52.50	68.53
5e-06	52.31	77.39
1e-05	53.86	76.01

0.0001	**54.39**	**87.04**
0.0005	**51.65**	**95.90**

==========

THIS IS PERFECT!!

1. Final Numbers

- **λ = 1e-6**
- **Test Accuracy = 52.50%**
- **Sparsity (<1e-2) = 68.53%**

Interpretation:

- **~69% of weights pruned**
- **Still maintaining >52% accuracy**

This is a solid accuracy–sparsity trade-off

---

2. Histogram (MOST IMPORTANT PART)

Left side (near 0)

- **Very large spike**
- **Most gates clustered near 0**

Strong pruning

Right side (near ~0.8–0.9)

- **Clear visible cluster**
- **Not many, but clearly separated**
- **Important weights preserved**

Middle region

- **Very thin distribution**

This is key:

- **Shows transition is minimal**
- **Model is making clear decisions**

<u>REQUIREMENT IN TASK:</u>

“A large spike at 0 and another cluster of values away from 0”

- **Big spike near 0**
- **Separate cluster near ~0.8–0.9**
- **Thin bridge (which is acceptable and expected)**

This is textbook self-pruning behavior

My model:

- **keeps important gates high (~0.8–0.9)**
- **pushes others near 0**
- **maintains accuracy**

That is true selective pruning

<u>ACHIEVEMENTS!!</u>

- **clear separation trend**
- **strong pruning**
- **preserved high-value gates**
- **good accuracy**
- **interpretable distribution**

WHY AN L1 PENALTY ON THE SIGMOID GATES ENCOURAGES SPARSITY

In normal loss function calculation, the aim is to increase prediction accuracy. The loss is calculated using “Cross Entropy”.
Eg. Cat 90%, Dog 50%, Bird 10%
Total Loss = Prediction Error
Then using “chain rule” in backpropagation, partial derivative of loss w.r.t to EVERY weight is calculated. Then, an optimiser like Adam is used to tweak the weights based on the results of the chain rule. This goes on iteratively for a lot of data until loss is reduced. This is how the network learns.

But in the self - pruning network, in the loss calculation we are also adding L1 loss. It is the sum of the gate_scores. It is “Sparsity Loss”.

$$\text{Total Loss} = \text{Classification Loss} + \lambda \cdot \sum g$$

We are not just aiming for prediction accuracy, but also for pruning the useless weights.
In my implementation right now, after i find the best model, i will add a hard pruning function that will actually remove the useless weights and even the tiny calculations for them need not be done to improve **performance.**
The goal of self-pruning is to improve performance.

$$\frac{\partial L}{\partial g} = \frac{\partial L_{cls}}{\partial g} + \lambda \cdot \frac{\partial L_{sparsity}}{\partial g}$$

This is how the gradients are calculated. The classification loss increases the gate values while the sparsity loss decreases them. In turn, a balance needs to be achieved